

TASK 27:ASSIGNMENTNO 27

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository: <https://github.com/anujaw193-star/python-project-collection>

1. (Creating Non-Linear Dataset)

Code:

```
[1] ✓ 1s  import pandas as pd
from sklearn.datasets import make_moons
|
X, y = make_moons(
    n_samples=500,
    noise=0.5,
    random_state=42
)
df = pd.DataFrame(X, columns=["Feature1", "Feature2"])

print("First 10 Rows:")
print(df.head(10))

print("\nShape of the dataset:", df.shape)
```

Output:

```
... First 10 Rows:
   Feature1  Feature2
0  0.831402 -0.107562
1  1.571772  1.003162
2  1.277576 -0.425889
3  0.210832  1.192568
4  0.630324  1.871216
5  1.357046 -0.605369
6  1.254682  0.068619
7  0.257022  1.530299
8  0.319506  0.322458
9  1.542390 -0.713281

Shape of the dataset: (500, 2)
```

2. (Scaling the Data)

Code:

```
[2]
✓ 0s from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)

X_scaled = pd.DataFrame(df_scaled, columns=["Feature1", "Feature2"])

print("\nFirst 5 Rows of Scaled Data:")
print(X_scaled.head())
#
```

Output:

```
...
First 5 Rows of Scaled Data:
   Feature1  Feature2
0  0.279933 -0.505782
1  1.004497  1.008551
2  0.716581 -0.939782
3 -0.327389  1.266783
4  0.083147  2.192034
```

3. (K-Means on Non-Linear Data)

Code:

```
[3]
✓ 1s from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

df["kmeans_cluster"] = clusters

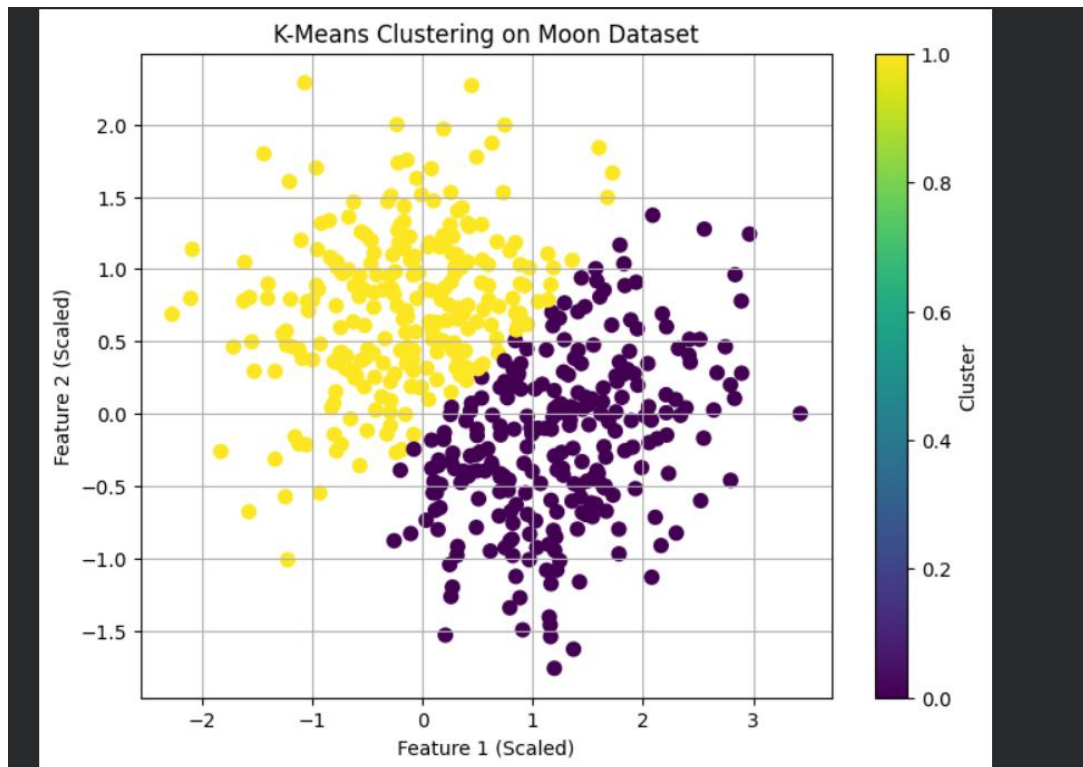
print(df.head())

plt.figure(figsize=(8, 6))
plt.scatter(df["Feature1"], df["Feature2"], c=df["kmeans_cluster"], cmap="viridis", s=50)

plt.title("K-Means Clustering on Moon Dataset")
plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
```

Output:

```
...
   Feature1  Feature2  kmeans_cluster
0  0.831402 -0.107562                0
1  1.571772  1.003162                0
2  1.277576 -0.425889                0
3  0.210832  1.192568                1
4  0.630324  1.871216                1
```



4. (DBSCAN Clustering)

Code:

```
[4]
✓ 1s
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

dbscan = DBSCAN(eps=0.3, min_samples=5)
dbscan_labels = dbscan.fit_predict(X_scaled)

df["dbscan_labels"] = dbscan_labels

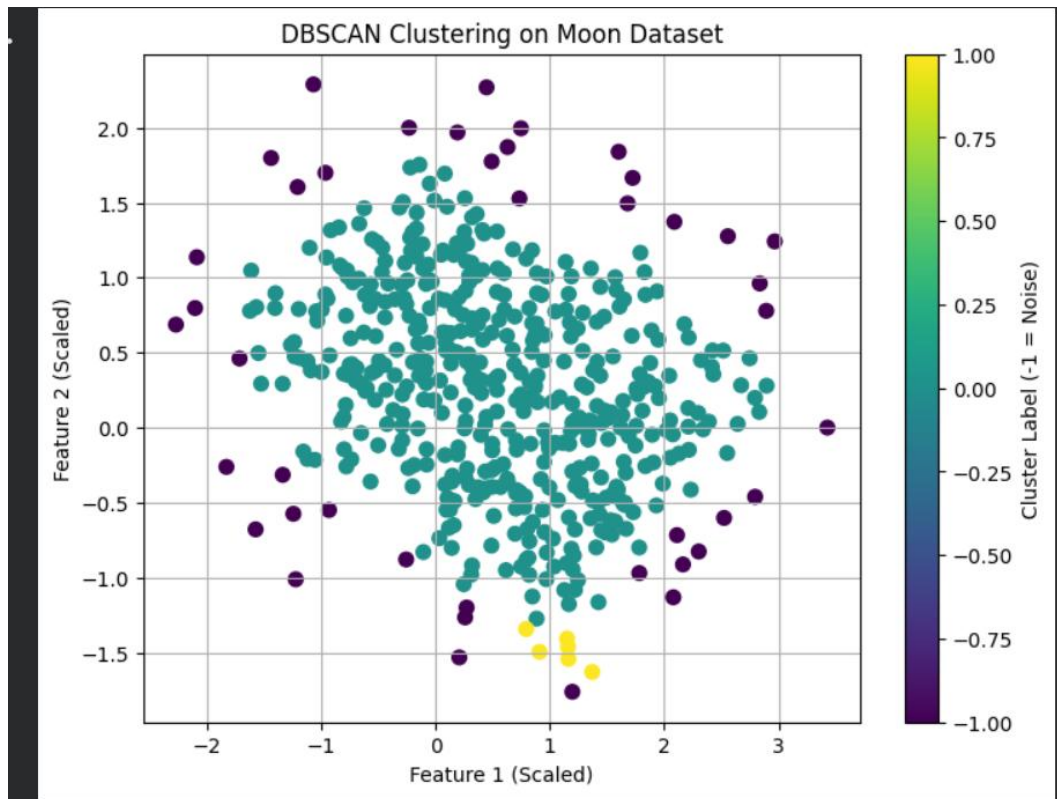
print(df.head())

plt.figure(figsize=(8, 6))
plt.scatter(df["Feature1"], df["Feature2"], c=df["dbscan_labels"], cmap="viridis", s=50)

plt.title("DBSCAN Clustering on Moon Dataset")
plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.colorbar(label="Cluster Label (-1 = Noise)")
plt.grid(True)
plt.show()
#
```

Output:

	...	Feature1	Feature2	kmeans_cluster	dbscan_labels
0		0.831402	-0.107562	0	0
1		1.571772	1.003162	0	0
2		1.277576	-0.425889	0	0
3		0.210832	1.192568	1	0
4		0.630324	1.871216	1	-1



5. (Comparing KMeans vs DBSCAN)

Code:

```
[7]
✓ Os ▶ plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(
    X_scaled["Feature1"],
    X_scaled["Feature2"],
    c=df["kmeans_cluster"],
    cmap="viridis",
    s=50
)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.grid(True)

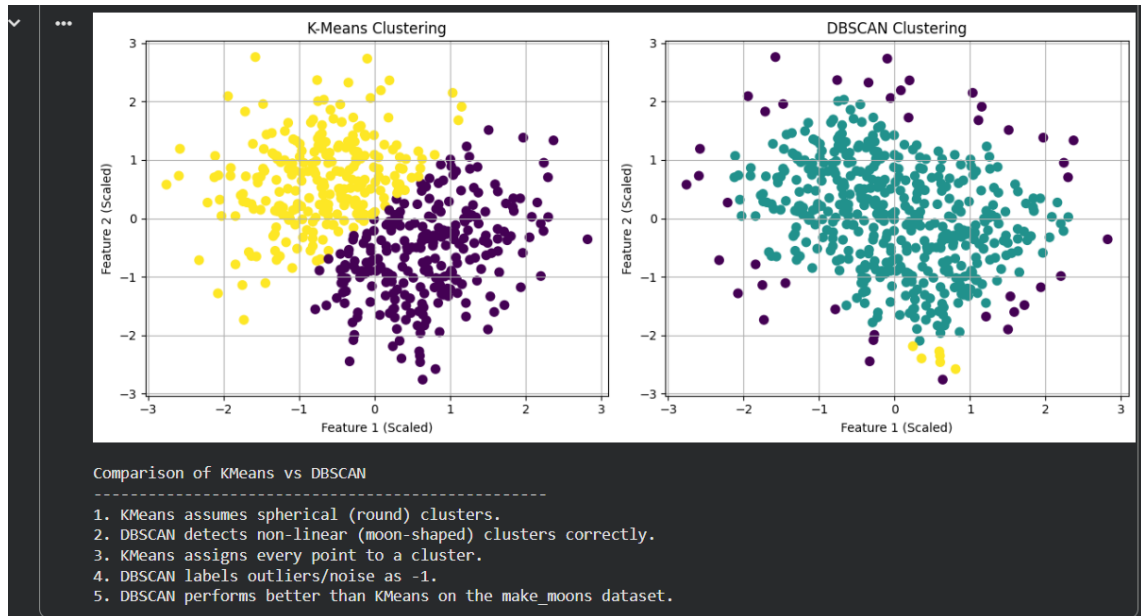
plt.subplot(1, 2, 2)
plt.scatter(
    X_scaled["Feature1"],
    X_scaled["Feature2"],
    c=df["dbscan_labels"],
    cmap="viridis",
    s=50
)
plt.title("DBSCAN Clustering")
plt.xlabel("Feature 1 (Scaled)")
plt.ylabel("Feature 2 (Scaled)")
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
plt.show()

print("\nComparison of KMeans vs DBSCAN")
print("-" * 50)
print("1. KMeans assumes spherical (round) clusters.")
print("2. DBSCAN detects non-linear (moon-shaped) clusters correctly.")
print("3. KMeans assigns every point to a cluster.")
print("4. DBSCAN labels outliers/noise as -1.")
print("5. DBSCAN performs better than KMeans on the make_moons dataset.")
```

Output:



6. (Tuning DBSCAN Parameters)

Code:

```
[7] 0s eps_values = [0.2, 0.3, 0.4, 0.5]
      results = []

      for eps in eps_values:
          dbscan = DBSCAN(eps=eps, min_samples=5)
          labels = dbscan.fit_predict(X_scaled)
          n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
          n_noise = list(labels).count(-1)
          results.append([eps, n_clusters, n_noise])

      results_df = pd.DataFrame(
          results,
          columns=["eps", "Number of Clusters", "Number of Noise Points"])

      print(results_df)
```

Output:

	eps	Number of Clusters	Number of Noise Points
0	0.2	4	116
1	0.3	2	42
2	0.4	1	25
3	0.5	1	9

7. (Creating High-Dimensional Dataset)

Code:

```
[8]
✓ Os
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

x, y = make_blobs(
    n_samples=500,
    n_features=6,
    centers=4,
    cluster_std=1.5,
    random_state=42
)
df = pd.DataFrame(
    x,
    columns=["Feature1", "Feature2", "Feature3", "Feature4", "Feature5", "Feature6"]
)
print("First 5 rows:")
print(df.head())

print("\nShape of Dataset:", df.shape)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
X_scaled = pd.DataFrame(X_scaled,
                        columns=df.columns)

print("\nFirst 5 rows of Scaled Data:")
print(X_scaled.head())
#
```

Output:

```
... First 5 rows:
   Feature1  Feature2  Feature3  Feature4  Feature5  Feature6
0  -7.430902  6.549456  2.166481  3.468039 -10.240054  8.934439
1  -0.746455 -5.467049  4.342113 -6.163268 -4.821673 -3.392003
2  -2.337856  9.239739  4.094461  1.887751 -6.417925 -9.445362
3  -7.659628  7.961709  0.571836  4.089885 -9.593714  7.660650
4 -10.169566  6.667335  3.105872  3.602202 -6.997864  8.798743

Shape of Dataset: (500, 6)

First 5 rows of Scaled Data:
   Feature1  Feature2  Feature3  Feature4  Feature5  Feature6
0  -1.052513  0.719083  0.350444  0.989657 -1.619597  1.414499
1   0.140795 -1.040620  0.841574 -0.857108  0.458731 -0.556293
2  -0.143302  1.113049  0.785668  0.686643 -0.153543 -1.524124
3  -1.093345  0.925894 -0.009533  1.108893 -1.371680  1.210841
4  -1.541419  0.736345  0.562503  1.015382 -0.375990  1.392803
```

8. (Applying PCA)

Code:

```
[10]
✓ Os
from sklearn.decomposition import PCA
import pandas as pd

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["Cluster"] = y

print("First 5 rows of PCA Data:")
print(pca_df.head())

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Explained Variance:")
print(pca.explained_variance_ratio_.sum())
#
```

Output:

```
... First 5 rows of PCA Data:
      PC1      PC2 Cluster
0  2.369765  1.259783      1
1 -0.943367 -0.829777      3
2  0.982191 -1.768556      0
3  2.265086  1.190377      1
4  2.142129  0.840804      1

Explained Variance Ratio:
[0.64327878 0.21039738]

Total Explained Variance:
0.853676159895889
```

9. (Visualizing PCA Results)

Code:

```
[11]
✓ Os
plt.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    c=pca_df["Cluster"],
    cmap="viridis",
    s=50
)

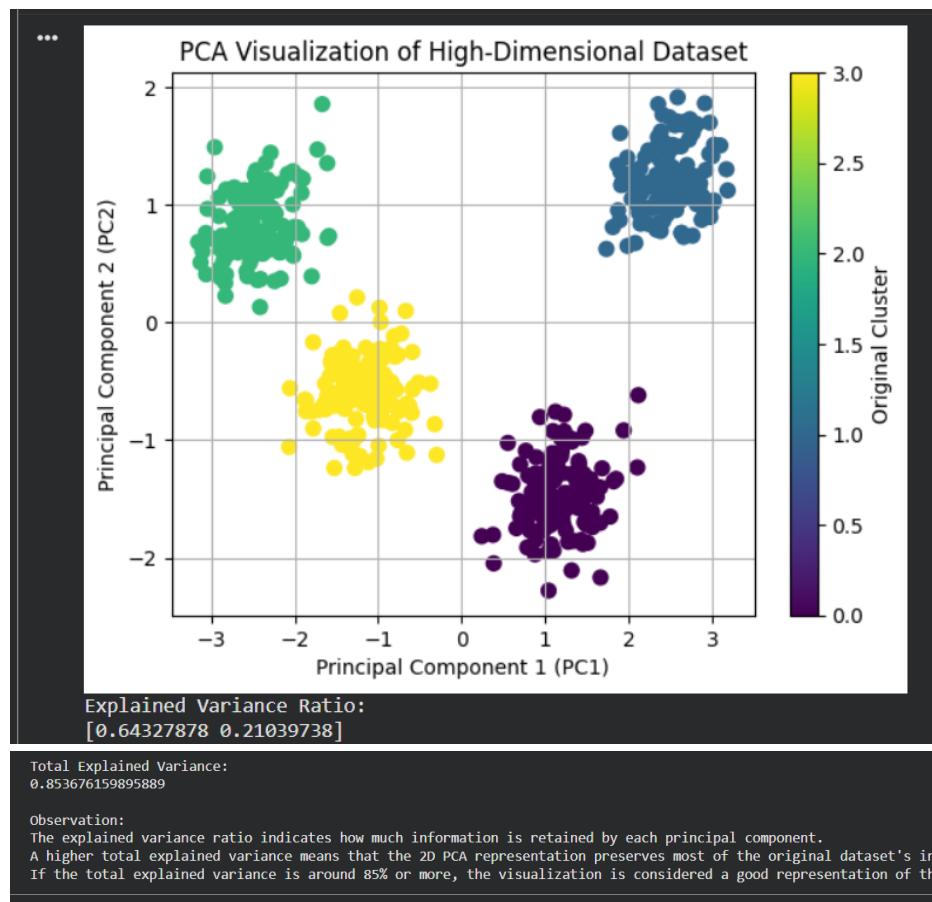
plt.title("PCA Visualization of High-Dimensional Dataset")
plt.xlabel("Principal Component 1 (PC1)")
plt.ylabel("Principal Component 2 (PC2)")
plt.colorbar(label="Original Cluster")
plt.grid(True)
plt.show()

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Explained Variance:")
print(pca.explained_variance_ratio_.sum())

print("\nObservation:")
print("The explained variance ratio indicates how much information is retained by each principal component.")
print("A higher total explained variance means that the 2D PCA representation preserves most of the original dataset's information.")
print("If the total explained variance is around 85% or more, the visualization is considered a good representation of the original 6-dimensional data.")
```

Output:



10. (Mini Project – Complete Pipeline)

Code:

```
[13] ✓ 1s from sklearn.decomposition import PCA

# =====
# Load Dataset and Select Numerical Features
# =====

wine_data = load_wine()
df = pd.DataFrame(wine_data.data, columns=wine_data.feature_names)
print("First 5 Rows:")
print(df.head())

print("\nShape:", df.shape)

# =====
# Scale the Data
# =====

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# =====
# Apply DBSCAN
# =====

dbscan = DBSCAN(eps=0.5, min_samples=5)
clusters = dbscan.fit_predict(X_scaled)
```

```
[13] ✓ 1s # =====
# Apply PCA (Reduce to 2 Components)
# =====

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["DBSCAN_Cluster"] = clusters

# =====
# Visualize DBSCAN Clusters in PCA Space
# =====

plt.figure(figsize=(8, 6))
plt.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    c=pca_df["DBSCAN_Cluster"],
    cmap="viridis",
    s=60
)
plt.title("DBSCAN Clusters on Wine Dataset(PCA) ")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label="Cluster (-1 = Noise)")
plt.grid(True)
plt.show()
```

```
# =====
# Explained Variance
# =====

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Explained Variance:")
print(pca.explained_variance_ratio_.sum())
#
```

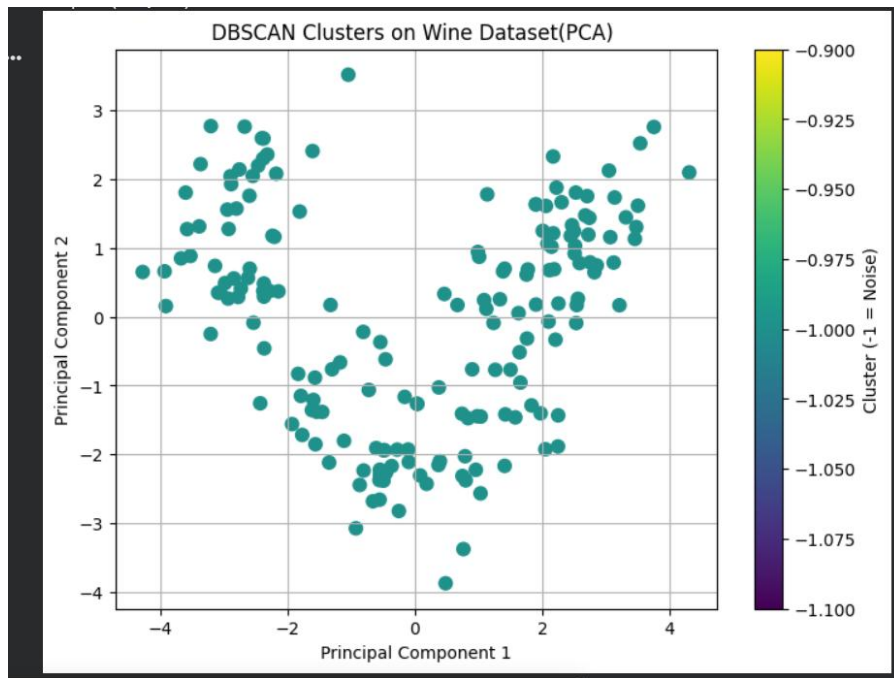
Output:

```
▼ ... First 5 Rows:
  alcohol  malic_acid  ash  alkalinity_of_ash  magnesium  total_phenols  \
0    14.23      1.71  2.43             15.6      127.0         2.80
1    13.20      1.78  2.14             11.2      100.0         2.65
2    13.16      2.36  2.67             18.6      101.0         2.80
3    14.37      1.95  2.50             16.8      113.0         3.85
4    13.24      2.59  2.87             21.0      118.0         2.80

  flavanoids  nonflavanoid_phenols  proanthocyanins  color_intensity  hue  \
0         3.06              0.28          2.29          5.64  1.04
1         2.76              0.26          1.28          4.38  1.05
2         3.24              0.30          2.81          5.68  1.03
3         3.49              0.24          2.18          7.80  0.86
4         2.69              0.39          1.82          4.32  1.04

  od280/od315_of_diluted_wines  proline
0              3.92      1065.0
1              3.40      1050.0
2              3.17      1185.0
3              3.45      1480.0
4              2.93       735.0

Shape: (178, 13)
```



Explained Variance Ratio:
[0.36198848 0.1920749]

Total Explained Variance:
0.5540633835693526